PROYECTO ACADÉMICO · OCTUBRE 2026

Intérprete educativo de pseudocódigo

ALGORA · Desarrollo de un lenguaje propio, análisis léxico, análisis sintáctico y ejecución controlada.

Estudiante	Gabriel Estuardo Polero Benavente
Carné	2890-24-8909
Carrera	Ingeniería en Sistemas de Información
Curso	Autómatas y Lenguajes Formales
Fecha	28 de octubre de 2026
Modalidad	Aplicación web local con Python, HTML, CSS y JavaScript

Informe de desarrollo y evidencia de las pruebas obligatorias

01 · Introducción

El aprendizaje de algoritmos requiere comprender cómo se escriben las instrucciones y cómo una computadora determina si están correctamente formuladas. ALGORA es una aplicación web local diseñada para apoyar a estudiantes que comienzan la carrera de Ingeniería en Sistemas de Información.

El sistema permite redactar pseudocódigo en español, analizarlo y ejecutarlo. La herramienta incluye una tabla de tokens, una consola con resultados y mensajes de error que indican la línea y columna del problema. El lenguaje se diseñó con reglas claras para que cada caso se pueda relacionar directamente con los contenidos de Autómatas y Lenguajes Formales.

Objetivo general

Desarrollar un intérprete educativo de pseudocódigo que aplique expresiones regulares, reconocimiento de tokens, gramáticas independientes de contexto y análisis sintáctico mediante descenso recursivo.

Objetivos específicos

Reconocer palabras reservadas, identificadores, números, cadenas, operadores y delimitadores. Verificar la estructura gramatical antes de ejecutar. Manejar variables, operaciones aritméticas, entrada/salida y condicionales Si/SiNo. Informar errores léxicos, sintácticos y de ejecución mediante mensajes comprensibles.

Tecnologías y arquitectura

Motor de lenguaje	Python 3, módulo re, clases de AST, parser descendente recursivo.
Servidor	Bibliotecas estándar http.server y json; no requiere instalaciones con pip.
Interfaz	HTML, CSS y JavaScript; editor, tokens, consola y captura de datos.
Pruebas	unittest: 20 pruebas automatizadas; 10 casos obligatorios documentados.

El intérprete nunca evalúa directamente el texto recibido como código Python. Primero construye un árbol sintáctico válido y únicamente interpreta las operaciones conocidas.

02 · Lenguaje y gramática

Sintaxis de ejemplo

```
Algoritmo Ejemplo
    Definir edad Como Entero;
    Leer(edad);
    Si edad >= 18 Entonces
        Escribir("Mayor de edad");
    SiNo
        Escribir("Menor de edad");
    FinSi
FinAlgoritmo
```

Elementos léxicos

El analizador usa expresiones regulares para clasificar palabras reservadas, identificadores, números enteros y decimales, cadenas de texto, operadores y símbolos. El lexer conserva la posición de cada token (línea y columna). Los comentarios // y espacios se ignoran.

Gramática independiente de contexto

Símbolo inicial: <programa>. Terminales: palabras reservadas, IDENTIFICADOR, NUMERO, CADENA, operadores + - * / = == != < <= > >= y signos ; , ( ). No terminales principales: programa, bloque, instrucción, expresión, término, factor y condición.

```
<programa>  ::= ALGORITMO ID <bloque> FINALGORITMO
<bloque>    ::= <instruccion> <bloque> | epsilon
<instr>     ::= <declaracion> | <asignacion>
             | <lectura> | <escritura> | <condicional>
<decl>      ::= DEFINIR ID COMO <tipo> ;
<tipo>      ::= ENTERO | DECIMAL | TEXTO
<asig>      ::= ID = <expr> ;
<leer>      ::= LEER ( ID ) ;
<escribir>  ::= ESCRIBIR ( <lista_exp> ) ;
<if>        ::= SI <condicion> ENTONCES <bloque>
                [SINO <bloque>] FINSI
```

La gramática BNF completa, incluyendo las producciones de precedencia y las cinco cadenas válidas e inválidas, se incluye en el documento GRAMATICA.md del repositorio.

02 · Gramática: expresiones

Producciones para expresiones y comparaciones

```
<condicion> ::= <expr> <relop> <expr>
<relop>    ::= == | != | < | <= | > | >=
<expr>     ::= <termino> <expr_r>
<expr_r>   ::= + <termino> <expr_r>
             | - <termino> <expr_r> | epsilon
<termino>  ::= <factor> <termino_r>
<termino_r>::= * <factor> <termino_r>
             | / <factor> <termino_r> | epsilon
<factor>   ::= NUMERO | CADENA | ID | ( <expr> )
             | + <factor> | - <factor>
<lista_exp>::= <expr> <lista_r>
<lista_r>  ::= , <expr> <lista_r> | epsilon
```

Relación con los temas del curso

Expresiones regulares	El lexer recorre el programa y reconoce patrones de tokens.
Autómatas finitos	Los identificadores se reconocen mediante un estado inicial, un estado de aceptación y transiciones para letras, dígitos y guion bajo.
GIC	Las producciones definen secuencias y anidamientos válidos de instrucciones.
Análisis sintáctico	El parser descendente recursivo transforma tokens en un árbol sintáctico (AST).
Ejecución	El intérprete evalúa el AST, guarda variables, procesa entradas y genera salidas.

La fase sintáctica se completa antes de ejecutar. Esto garantiza que un programa con errores gramaticales no produzca resultados parciales.

03 · Evidencia del análisis léxico

Programa analizado: AritmeticaBasica. La siguiente tabla es una muestra de tokens obtenida directamente del motor y representa palabras reservadas, identificadores, delimitadores y operadores. La lista íntegra se incluye en PRUEBAS.md.

LEXEMA	TIPO	LÍNEA	COLUMNA
Algoritmo	ALGORITMO	1	1
AritmeticaBasica	IDENTIFICADOR	1	11
Definir	DEFINIR	2	5
a	IDENTIFICADOR	2	13
Como	COMO	2	15
Entero	ENTERO	2	20
;	;	2	26
Definir	DEFINIR	3	5
b	IDENTIFICADOR	3	13
Como	COMO	3	15
Entero	ENTERO	3	20
;	;	3	26
Definir	DEFINIR	4	5
resultado	IDENTIFICADOR	4	13
Como	COMO	4	23
Entero	ENTERO	4	28
;	;	4	34
a	IDENTIFICADOR	5	5
=	=	5	7

Total identificado en este programa: 41 tokens. La posición de cada token permite señalar con precisión dónde se produce un error.

04 · Prueba obligatoria 01 / 10

Asignación y precedencia aritmética

Código ingresado

```
Algoritmo AritmeticaBasica
    Definir a Como Entero;
    Definir b Como Entero;
    Definir resultado Como Entero;
    a = 10;
    b = 4;
    resultado = a + b * 2;
    Escribir("Resultado: ", resultado);
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	No aplica
Resultado esperado	Resultado: 18
Resultado obtenido	Resultado: 18
Clasificación	Ejecución correcta

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 02 / 10

Paréntesis y números decimales

Código ingresado

```
Algoritmo ParentesisYDecimales
    Definir precio Como Decimal;
    Definir total Como Decimal;
    precio = 12.5;
    total = (precio + 2.5) / 3;
    Escribir("Total: ", total);
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	No aplica
Resultado esperado	Total: 5
Resultado obtenido	Total: 5
Clasificación	Ejecución correcta

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 03 / 10

Entrada de número entero

Código ingresado

```
Algoritmo EntradaEdad
    Definir edad Como Entero;
    Leer(edad);
    Escribir("Edad ingresada: ", edad);
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	21
Resultado esperado	Edad ingresada: 21
Resultado obtenido	Edad ingresada: 21
Clasificación	Ejecución correcta

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 04 / 10

Entrada y salida de texto

Código ingresado

```
Algoritmo EntradaNombre
    Definir nombre Como Texto;
    Leer(nombre);
    Escribir("Hola, ", nombre, "!");
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	Ana
Resultado esperado	Hola, Ana!
Resultado obtenido	Hola, Ana!
Clasificación	Ejecución correcta

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 05 / 10

Condición verdadera

Código ingresado

```
Algoritmo MayorDeEdad
    Definir edad Como Entero;
    edad = 20;
    Si edad >= 18 Entonces
        Escribir("Es mayor de edad");
    SiNo
        Escribir("Es menor de edad");
    FinSi
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	No aplica
Resultado esperado	Es mayor de edad
Resultado obtenido	Es mayor de edad
Clasificación	Ejecución correcta

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 06 / 10

Condición falsa

Código ingresado

```
Algoritmo CondicionFalsa
    Definir nota Como Decimal;
    nota = 49.5;
    Si nota >= 61 Entonces
        Escribir("Aprobado");
    SiNo
        Escribir("Reprobado");
    FinSi
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	No aplica
Resultado esperado	Reprobado
Resultado obtenido	Reprobado
Clasificación	Ejecución correcta

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 07 / 10

Detección de símbolo inválido

Código ingresado

```
Algoritmo CaracterInvalido
    Definir valor Como Entero;
    valor = 4 @ 2;
    Escribir(valor);
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	No aplica
Resultado esperado	Error léxico, línea 3, carácter @
Resultado obtenido	Error lexico en linea 3, columna 15: Caracter no permitido: '@'.
Clasificación	Error lexico

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 08 / 10

Falta de punto y coma

Código ingresado

```
Algoritmo PuntoYComa
    Definir x Como Entero;
    x = 7
    Escribir(x);
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	No aplica
Resultado esperado	Error sintáctico, línea 4, falta ;
Resultado obtenido	Error sintactico en linea 4, columna 5: Falta ; al finalizar la asignacion (se encontro 'Escribir').
Clasificación	Error sintactico

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 09 / 10

Control de división entre cero

Código ingresado

```
Algoritmo DivisionCero
    Definir resultado Como Decimal;
    resultado = 5 / (3 - 3);
    Escribir(resultado);
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	No aplica
Resultado esperado	Error de ejecución: división entre cero
Resultado obtenido	Error de ejecucion en linea 3, columna 19: Division entre cero. Revise el denominador.
Clasificación	Error de ejecucion

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

04 · Prueba obligatoria 10 / 10

Control de variable no declarada

Código ingresado

```
Algoritmo VariableNoDeclarada
    total = 5;
    Escribir(total);
FinAlgoritmo
```

Registro de ejecución

Datos de entrada	No aplica
Resultado esperado	Error de ejecución: variable no declarada
Resultado obtenido	Error de ejecucion en linea 2, columna 5: La variable 'total' no esta declarada. Declarela con Definir.
Clasificación	Error de ejecucion

ESTADO: APROBADA

Evidencia: resultado registrado automáticamente al invocar el motor real mediante el script documentacion/generar_registros.py. El código fuente del caso está en la carpeta ejemplos/.

05 · Repositorio y entrega

Repositorio de GitHub

Enlace público: pendiente de publicar. Debe sustituirse por la URL real de la cuenta del estudiante antes de entregar este PDF.

El proyecto está preparado para publicarse en GitHub y contiene código fuente, README con pasos de instalación, documentación gramatical, programas de ejemplo, batería de pruebas y registros de las ejecuciones.

Instrucciones básicas

```
python app.py
# Abrir http://localhost:8000

python -m unittest discover -s tests -v
```

06 · Conclusiones

El desarrollo permitió relacionar los conceptos de Autómatas y Lenguajes Formales con una aplicación funcional. Las expresiones regulares identifican las unidades del lenguaje y la gramática define cómo deben combinarse. El análisis sintáctico convierte esas unidades en estructuras que pueden interpretarse de manera controlada.

El uso de variables, operaciones con precedencia, entrada de datos y condiciones permite ejecutar algoritmos útiles para estudiantes de primer ingreso. Los mensajes de error con ubicación ayudan a explicar la diferencia entre errores léxicos, sintácticos y de ejecución.

Las diez pruebas obligatorias registradas fueron aprobadas, al igual que otras diez verificaciones automáticas. Para completar la entrega final únicamente resta publicar el repositorio y reemplazar el campo del enlace de GitHub en este informe.

Referencias de implementación: motor.py, tests/test_interprete.py, documentacion/GRAMATICA.md y documentacion/PRUEBAS.md.